

BERRI
TXARRAK



css Nesting

What is CSS Nesting?

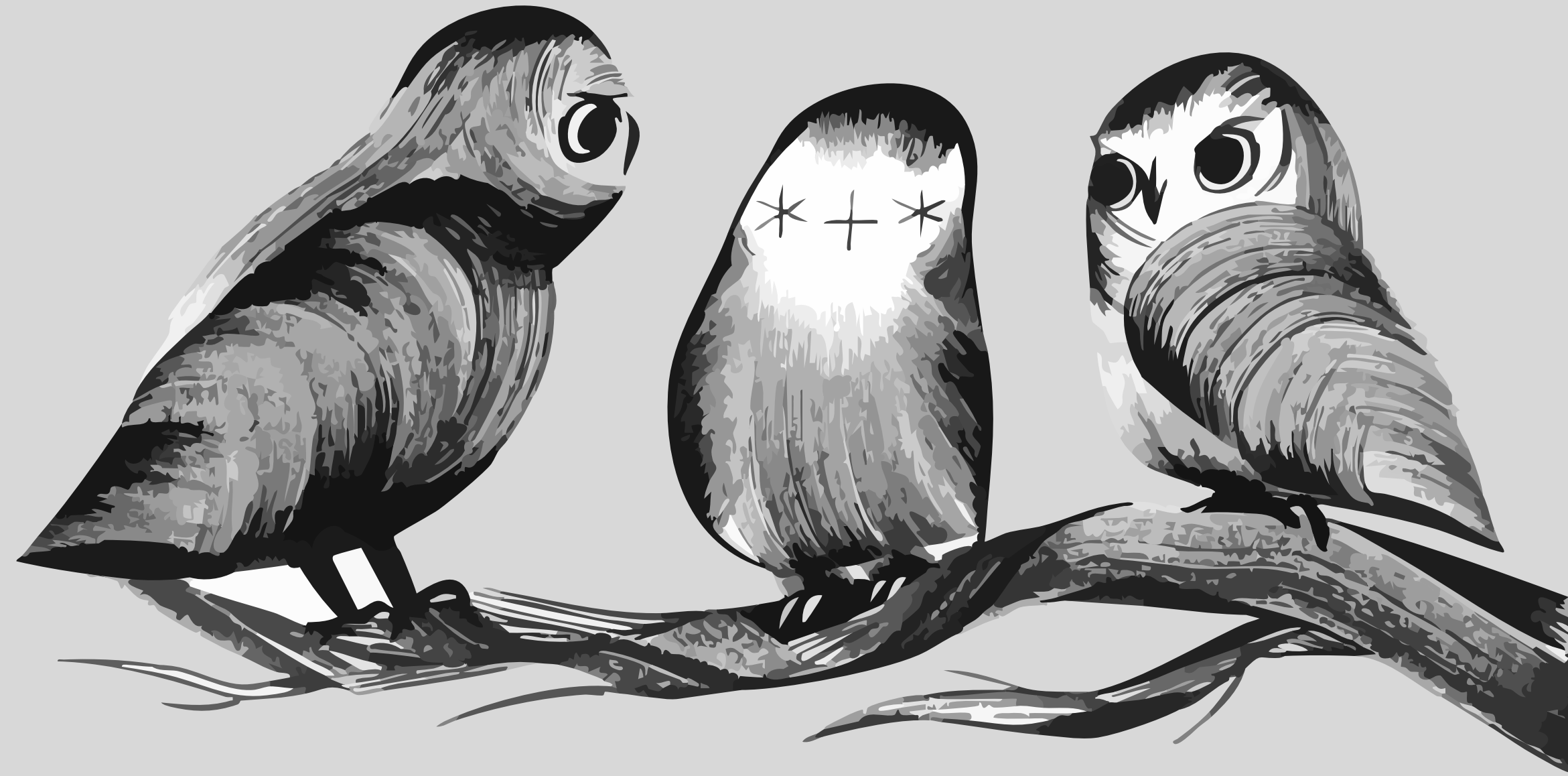
A native CSS feature that lets you nest selectors

Write child styles inside parent selectors

Hierarchical structure that mirrors your HTML

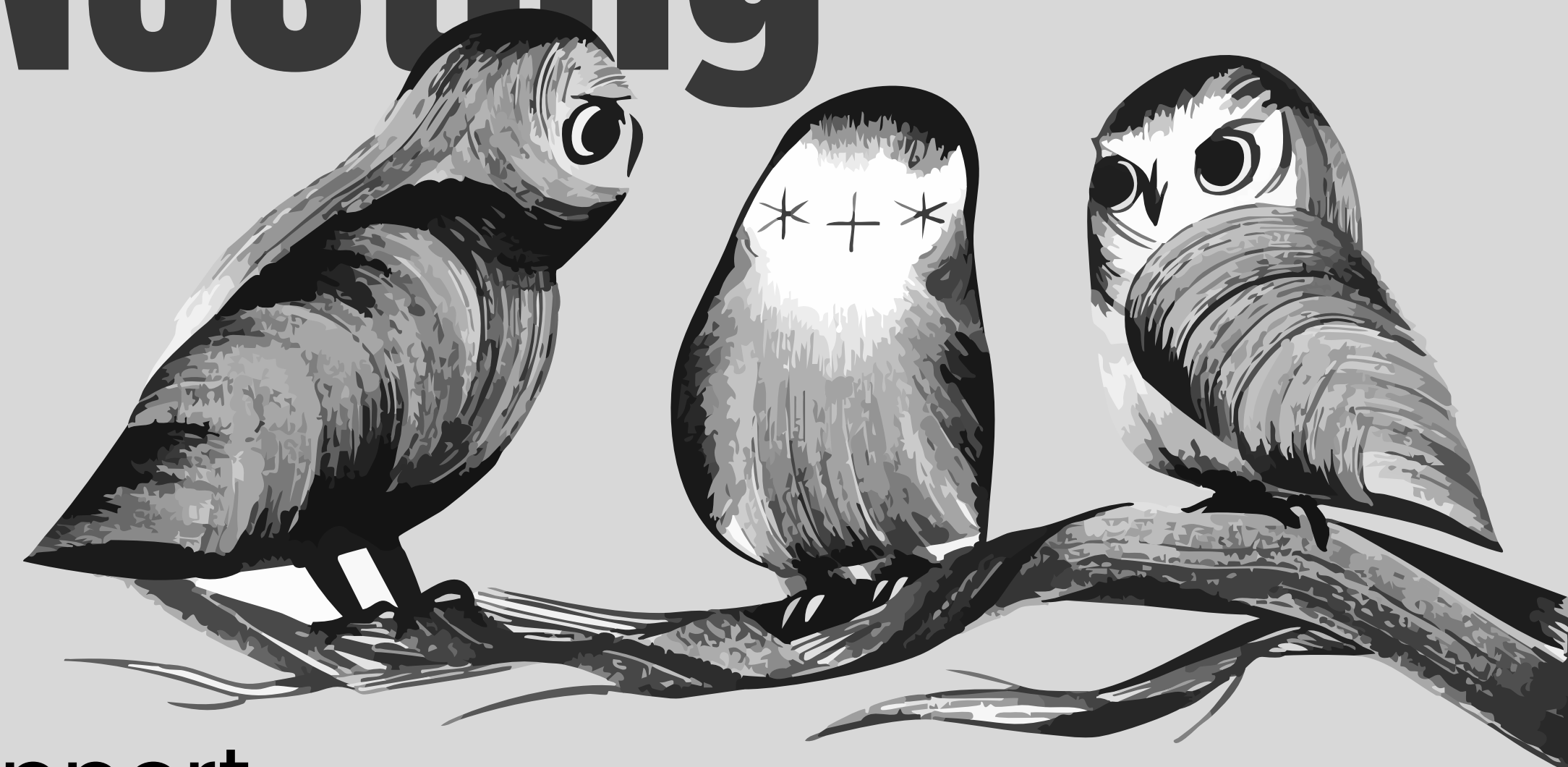


CSS nesting is a feature that allows us to write CSS selectors inside other selectors, creating a hierarchical structure that mirrors our HTML. Instead of writing repetitive selectors, we can group related styles together in a logical, nested format.



A BRIEF HISTORY

Where Did CSS Nesting Come From?



2006: Sass created by Hampton Catlin

2009: Less CSS released with nesting support

2006–2023: Developers used preprocessors for nesting

2023: CSS nesting becomes native in browsers

– *Why was it used? To avoid repetitive code and improve maintainability*

Sass (Syntactically Awesome Style Sheets)

Back in 2006, Hampton Catlin created Sass – a CSS preprocessor that introduced nesting amongst other features. Soon after, Less CSS followed with similar capabilities. For nearly 17 years, developers relied on these preprocessors to write nested CSS, which would then be compiled into traditional CSS for browsers. Why did developers love it so much that they were willing to add build tools to their workflow? Because it solved a fundamental problem: repetitive, hard-to-maintain CSS. The nesting feature was so popular that it became one of the main reasons developers adopted Sass or Less. Now, in 2023, CSS nesting has finally become native – browsers understand it directly without any compilation step. Let me show you exactly what problem this solves with some real examples..

Navigation Example

```
header {  
  background: #333;  
  padding: 1rem;  
}  
  
header nav {  
  display: flex;  
  gap: 2rem;  
}  
  
header nav ul {  
  list-style: none;  
  display: flex;  
}  
  
header nav a {  
  colour: white;  
  text-decoration: none;  
}  
  
header nav a:hover {  
  colour: #3498db;  
}
```

Before
(Traditional
CSS)

```
header {  
  background: #333;  
  padding: 1rem;  
}  
  
nav {  
  display: flex;  
  gap: 2rem;  
}  
  
ul {  
  list-style: none;  
  display: flex;  
}  
  
a {  
  colour: white;  
  text-decoration: none;  
  
  &:hover {  
    colour: #3498db;  
  }  
}  
}
```

After (CSS
Nesting)

"Here's the problem CSS nesting solves. On the left, traditional CSS requires us to repeat 'header' and 'header nav' multiple times. We write header, then header nav, then header nav ul, then header nav a, then header nav a:hover. Notice the repetition? If we later rename header to 'site-header', we'd need to find and replace it in five different places. On the right, with nesting, all header-related styles are in one logical block. We write 'header' once, and everything inside is automatically scoped to it. If we need to rename it, we only change it in one place. Also notice the ampersand (&) before :hover – this is crucial. The & represents the parent selector, so &:hover becomes 'a:hover'. This is the key to understanding CSS nesting."

THE & NESTING SELECTOR

& = represents the parent selector

&:hover applies hover state to parent

&:focus applies focus state to parent

Without &, creates descendant selector

Always use & for pseudo-classes and pseudo-elements

"The ampersand is the heart of CSS nesting. It's a special character that represents the parent selector. Look at this article example below.

```
article {
  padding: 2rem;

  /* Creates: article h1 */
  h1 {
    font-size: 2rem;
  }

  /* Creates: article:hover */
  &:hover {
    box-shadow: 0 2px 8px rgba(0,0,0,0.1);
  }

  /* Creates: article::before */
  &::before {
    content: " 📄 ";
  }
}
```

When we write 'h1' without the ampersand, it creates a descendant selector: 'article h1' – selecting h1 elements inside article. But when we write '&:hover', the ampersand is replaced with 'article', creating 'article:hover' – applying the hover state to the article itself, not its descendants. The same applies to ::before – '&::before' becomes 'article::before'. This is essential: always use & before pseudo-classes like :hover, :focus, :active, and pseudo-elements like ::before, ::after. Without it, the nesting won't work as you expect. Now let's look at practical use cases..."

SEMANTIC HTML STRUCTURE

Problem it solves:

Repetitive parent
selectors scattered
across your stylesheet

```
section {  
  padding: 3rem 1rem;  
  
  header {  
    margin-bottom: 2rem;  
  
    h2 {  
      font-size: 2.5rem;  
      colour: #2c3e50;  
    }  
  
    p {  
      colour: #7f8c8d;  
      font-size: 1.1rem;  
    }  
  }  
  
  article {  
    margin-bottom: 1.5rem;  
    line-height: 1.6;  
  
    &:last-child {  
      margin-bottom: 0;  
    }  
  }  
}
```

"This is a perfect example of the problem CSS nesting solves: organisation. Imagine you're styling a section with a header and multiple articles. Traditionally, you'd write: section, then section header, then section header h2, then section header p, then section article, then section article:last-child. These selectors end up scattered throughout your CSS file. The problem? When you need to modify anything related to the section, you're hunting through your entire stylesheet. With nesting, everything related to the section is in one place. All header styles are nested inside section, all article styles are together. If I need to change how the section works, I come to this one block. Notice how &:last-child removes the bottom margin from the last article – the ampersand is essential here because we're targeting the article itself, not its children."

RESPONSIVE MEDIA QUERIES

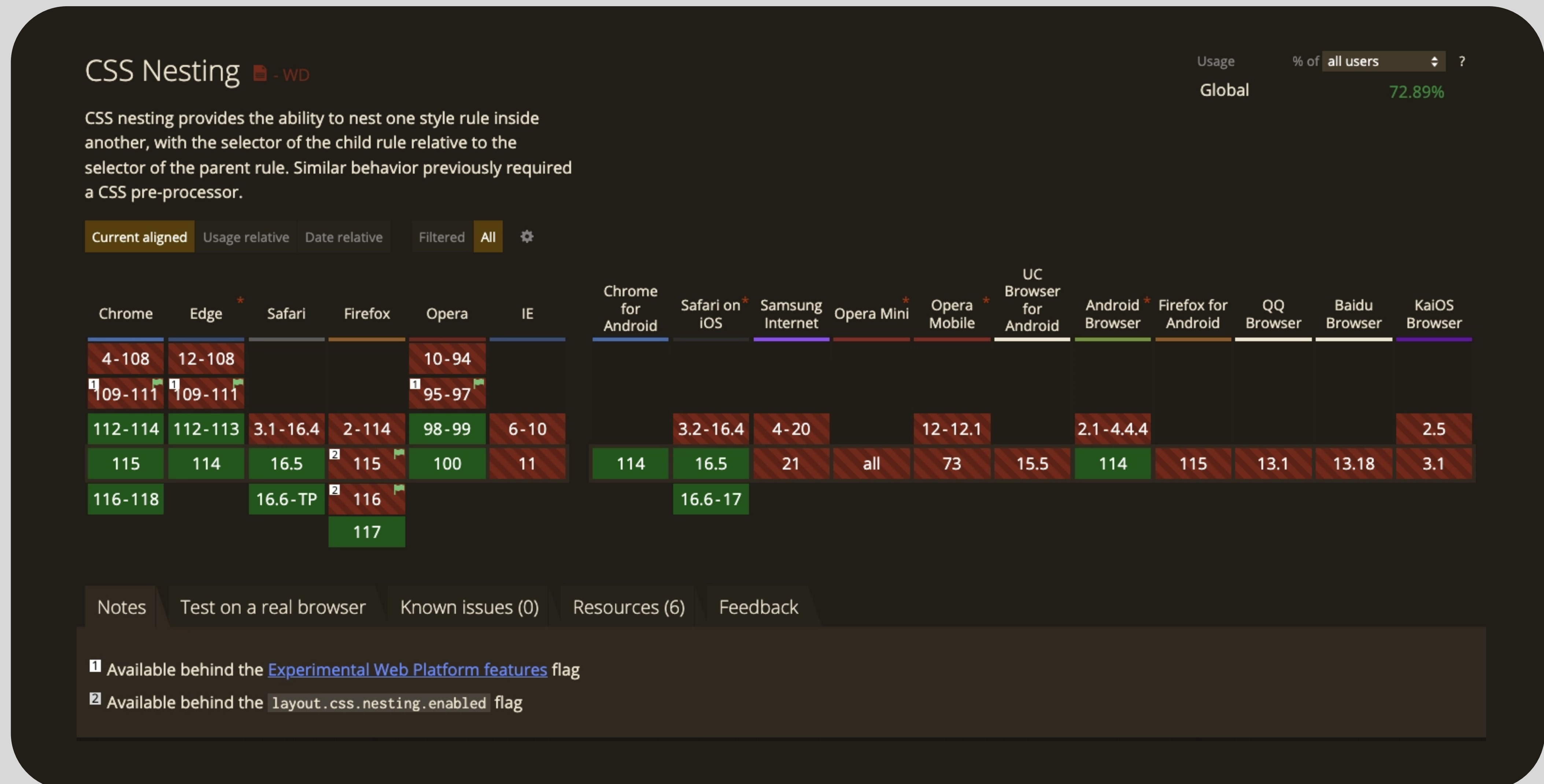
Problem it solves: Media queries scattered throughout your CSS, hard to see component's responsive behaviour

```
article {  
  padding: 1rem;  
  
  h1 {  
    font-size: 1.5rem;  
    margin-bottom: 1rem;  
  }  
  
  p {  
    font-size: 1rem;  
    line-height: 1.5;  
  }  
  
  @media (min-width: 768px) {  
    padding: 2rem;  
  
    h1 {  
      font-size: 2.5rem;  
    }  
  
    p {  
      font-size: 1.125rem;  
      line-height: 1.7;  
    }  
  }  
}
```

```
@media (min-width: 1024px) {  
  padding: 3rem;  
  max-width: 800px;  
  margin: 0 auto;  
}
```

Here's another major problem that nesting solves: responsive media queries. In traditional CSS, we'd have our base article styles at the top, then scroll way down to a @media query block with more article styles, then scroll further to another @media block. The problem? You can't see at a glance how the article responds across different screen sizes. You're jumping between different parts of your file. With nesting, the responsive behaviour is right there with the base styles. I can immediately see: on mobile, padding is 1rem and heading is 1.5rem; on tablets, padding increases to 2rem and heading to 2.5rem; on desktops, padding is 3rem and we centre it with a max-width. Everything about this article's responsive design is in one logical place. This makes it so much easier to maintain and modify responsive behaviour."

BROWSER SUPPORT



After 17 years of being available only through preprocessors, CSS nesting finally became native in 2023. All major browsers now support it. Chrome and Edge got support in April 2023, Safari in May 2023, and Firefox in August 2023. According to Can I Use, we now have over 90% global browser support. For modern web projects, internal tools, or applications with defined browser requirements, CSS nesting is production-ready today."

WHEN TO USE CSS NESTING

Perfect for:

- Modern web applications (2023+ browsers)
- Internal tools and dashboards
- Projects with defined browser requirements
- Component-based architectures

Be cautious with:

- Clients requiring older browser support
- Public-facing sites with diverse audiences
- Use PostCSS as a fallback solution

CSS nesting is perfect for modern web applications and internal tools where you control the browser environment. However, be cautious with public-facing websites serving diverse audiences. If you need older browser support, use PostCSS with the `postcss-nesting` plugin during your build process – it automatically converts nested CSS to traditional CSS for older browsers."

Always use

& for pseudo-classes (:hover, :focus, :active)
& for pseudo-elements (::before, ::after)

Don't nest too deeply (max 3-4 levels)

Use nesting for related styles, not everything

Keep specificity manageable

Test across all target browsers

Follow these best practices: Always use the ampersand for pseudo-classes and pseudo-elements – this is essential. Don't nest more than 3-4 levels deep as it creates high specificity that's hard to override. Use nesting where it improves organisation, not just because you can. And always test across your target browsers to ensure compatibility."

The biggest mistake is nesting too deeply. Five levels deep creates selectors like 'main section article header h1' which is incredibly specific and hard to override. Use flatter nesting. Another common mistake is forgetting the ampersand before pseudo-classes – without it, your styles won't work as expected. Finally, don't nest everything – only nest where it improves organisation.

COMMON PITFALLS

Too Deeply Nested:

```
main {  
  section {  
    article {  
      header {  
        h1 {  
          /* 5 levels - too much! */  
        }  
      }  
    }  
  }  
}
```

Better Approach:

```
main {  
  section {  
    padding: 2rem;  
  }  
  
  article {  
    margin-bottom: 1rem;  
  }  
  
  h1 {  
    font-size: 2rem;  
  }  
}
```

The biggest mistake is nesting too deeply. Five levels deep creates selectors like 'main section article header h1' which is incredibly specific and hard to override. Use flatter nesting. Another common mistake is forgetting the ampersand before pseudo-classes – without it, your styles won't work as expected. Finally, don't nest everything – only nest where it improves organisation.

BERRI
TXARRAK



The end.....
